../media/sdu-black-eps-converted-to.pdf

Concurrent Programming

Week 10 (Lecture 4)

Stelios Tsampas

March 3, 2026

stelios@imada.sdu.dk

https://www.steliostsampas.com

stelios-tau/cp-2026

Outline

Recap and introduction

Last week's topics

Last week, we touched upon...

- The fundamental problem of sharing mutable state.
- Atomicity, i.e. the property of a sequence of statements appearing *indivisible* w.r.t. other threads.
- How to achieve atomicity and protect code against concurrent access by other threads using *intrinsic locks*.
- Several examples of race conditions and how to fix them.

Last week vs this week

Last week was more about specific problematic patterns occurring during the execution of a *program*.

Last week vs this week

Last week was more about specific problematic patterns occurring during the execution of a *program*.

This week will be more about *designing* software that can be *utilized* safely in a concurrent setting.

This week, we will witness that

- Java subverts common sense on a concurrent setting, due to *visibility*.
- Preparing to *share* an object across threads is not as simple as it sounds, due to timing and visibility problems.
- Notions such as *invariants*, *immutability* and *encapsulation* play a central role in designing thread-safe classes.

This week's topics

This week, we will witness that

- Java subverts common sense on a concurrent setting, due to *visibility*.
- Preparing to *share* an object across threads is not as simple as it sounds, due to timing and visibility problems.
- Notions such as *invariants*, *immutability* and *encapsulation* play a central role in designing thread-safe classes.

We will also demonstrate all of the above in examples.

Visibility

More than just locking

Intrinsic locks ensure that only one thread may enter any piece of code that is protected by the same lock.

It also has a second purpose: it makes updates in one thread *visible* to other threads!

More than just locking

Intrinsic locks ensure that only one thread may enter any piece of code that is protected by the same lock.

It also has a second purpose: it makes updates in one thread *visible* to other threads!

> Without synchronization, this might not happen!

Is this thread-safe?


```
1  class KeepsGoing extends Thread {
2      boolean keepRunning = true;
3
4      public void run() {
5          while (keepRunning) {
6              doSomething();
7          }
8      }
9  }
```

Is this thread-safe?


```
10 class KeepsGoing extends Thread {
11     boolean keepRunning = true;
12
13     public void run() {
14         while (keepRunning) {
15             doSomething();
16         }
17     }
18 }
```

Is this thread-safe?


```
19 class KeepsGoing extends Thread {
20     boolean keepRunning = true;
21
22     public void run() {
23         while (keepRunning) {
24             doSomething();
25         }
26     }
27 }
```

There is no guarantee that changes to `keepRunning` by other threads will be visible in this thread.

We have to use synchronization!

The developer might expect that writes should be *visible* to any access that took place *after* (in terms of universal time) a write.

However, this is **not** the case:

> Java does not guarantee when a write in some thread will be visible to other threads.

The culprit here are optimization taking place by Java. They are only correctness-preserving in a single-threaded environment.

The developer might expect that writes should be *visible* to any access that took place *after* (in terms of universal time) a write.

However, this is **not** the case:

> Java does not guarantee when a write in some thread will be visible to other threads.
> Java doesn't even guarantee that writes will be visible **at all**!!

The culprit here are optimization taking place by Java. They are only correctness-preserving in a single-threaded environment.

Stale data

Accessing out-of-date data due to the state of data (i.e. a variable being written upon) not being up-to-date across different threads. May lead to serious correctness issues.

Visibility and stale data

Stale data

Accessing out-of-date data due to the state of data (i.e. a variable being written upon) not being up-to-date across different threads. May lead to serious correctness issues.

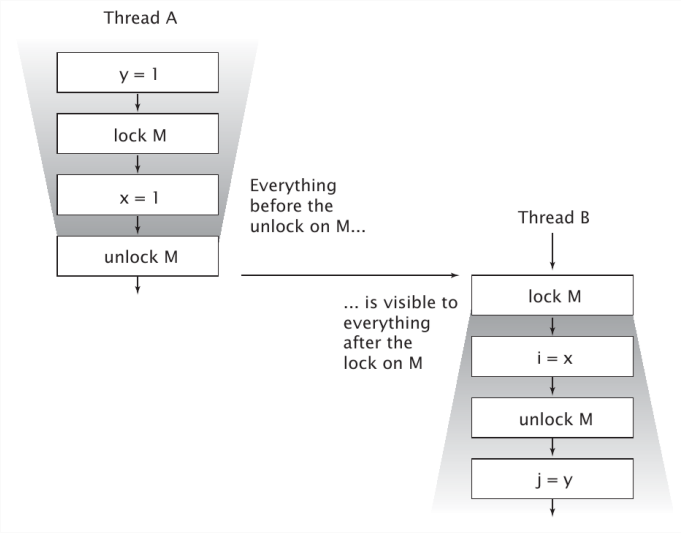
Solution

Always use synchronization to access shared data!

Synchronization causes writes to be visible across threads!

Visibility

../media/sdu-



Is this thread-safe (2)


```
28 public class MutableInteger {
29     private int value;
30
31 public int get() { return value; }
32 public void set(int value) { this.value = value; }
33 }
```

Is this thread-safe (2)


```
40 public class MutableInteger {
41     private int value;
42
43 public int get() { return value; }
44 public void set(int value) { this.value = value; }
45 }
```

Is this thread-safe (2)


```
52 public class MutableInteger {
53     private int value;
54
55 public int get() { return value; }
56 public void set(int value) { this.value = value; }
57 }
```

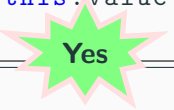

```
58 public class MutableInteger {
59     private int value;
60
61 public int get() { return value; }
62 public synchronized void set(int value) { this.value = value; }
63 }
```

Is this thread-safe (2)


```
64 public class MutableInteger {
65     private int value;
66
67 public synchronized int get() { return value; }
68 public synchronized void set(int value) { this.value = value; }
69 }
```

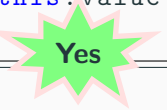
Is this thread-safe (2)


```
70 public class MutableInteger {
71     private int value;
72
73 public synchronized int get() { return value; }
74 public synchronized void set(int value) { this.value = value; }
75 }
```



Is this thread-safe (2)


```
76 public class MutableInteger {
77     private int value;
78
79 public synchronized int get() { return value; }
80 public synchronized void set(int value) { this.value = value; }
81 }
```



The moral of the story is, intrinsic locks are not just for synchronization, but also for **visibility**. Use them when accessing shared variables, regardless if there are any apparent race conditions or not.

Out-of-thin-air safety

Staleness is not all-or-nothing

> One value might be fresh while another might be stale.

There is **almost** one thing that the developer can be sure when accessing stale data.

> Any value accessed to a shared memory location was caused by a write, and did not appear out of nowhere.
>
> This is known as *out-of-thin-air* safety.

Out-of-thin-air safety

Staleness is not all-or-nothing

> One value might be fresh while another might be stale.

There is **almost** one thing that the developer can be sure when accessing stale data.

> Any value accessed to a shared memory location was caused by a write, and did not appear out of nowhere.
>
> This is known as *out-of-thin-air* safety.

It applies to **almost** any variable out there.

Out-of-thin-air safety

Staleness is not all-or-nothing

> One value might be fresh while another might be stale.

There is **almost** one thing that the developer can be sure when accessing stale data.

> Any value accessed to a shared memory location was caused by a write, and did not appear out of nowhere.
> This is known as *out-of-thin-air* safety.

It applies to **almost** any variable out there.

Except `double`'s and `long`'s.

Out-of-thin-air safety

Staleness is not all-or-nothing

One value might be fresh while another might be stale.

There is **almost** one thing that the developer can be sure when accessing stale data.

Any value accessed to a shared memory location was caused by a write, and did not appear out of nowhere.
This is known as *out-of-thin-air* safety.

It applies to **almost** any variable out there.

Except `double`'s and `long`'s.

By the way, remember DataRace.java?

`Volatile` variables

The `volatile` modifier ensures that an attribute's value is always the same when read from any thread.

One sneaky way to ensure visibility is to use `volatile` variables.

They achieve synchronization, as Java ensures that a read to a volatile variable will *always* return the most recent value.

They are neat and lightweight.

Like locks, they also ensure that all variables prior to reading/writing a volatile variable are *visible* across threads.

Remember!

Locking can guarantee both visibility and atomicity; volatile variables can only guarantee visibility.

You should use volatile variables only when all the following criteria are met:

- Writes to the variable do not depend on its current value, or you can ensure that only a single thread ever updates the value;
- The variable does not participate in invariants with other state variables, and
- Locking is not required for any other reason while the variable is being accessed.

Publishing and sharing

Publishing an object

You publish an object when it becomes available outside its scope, e.g. by passing its reference around.

It is easy to inadvertently publish an object, so it is important to understand when publishing happens.

There are also various pitfalls associated with publishing in a concurrent setting.

Publishing an object

You publish an object when it becomes available outside its scope, e.g. by passing its reference around.

It is easy to inadvertently publish an object, so it is important to understand when publishing happens.

> We say that such objects have *escaped*.

There are also various pitfalls associated with publishing in a concurrent setting.

Publishing an object

Publishing an object can be as simple as using a public variable.

```
82 public static Set<Secret> knownSecrets;
83 public void initialize() {
84     knownSecrets = new HashSet<Secret>();
85 }
```

Publishing an object

Publishing an object can be as simple as using a public variable.

```
91 public static Set<Secret> knownSecrets;
92 public void initialize() {
93     knownSecrets = new HashSet<Secret>();
94 }
```

Or returning an object.

```
95 class UnsafeStates {
96     private String[] states =
97         new String[] {"AK", "AL" ...};
98     public String[] getStates() { return states; }
99 }
```

Publishing an object

Publishing an object can be as simple as using a public variable.

```
100 public static Set<Secret> knownSecrets;
101 public void initialize() {
102     knownSecrets = new HashSet<Secret>();
103 }
```

Or returning an object.

```
104 class UnsafeStates {
105     private String[] states =
106         new String[] {"AK", "AL" ...};
107     public String[] getStates() { return states; }
108 }
```

Publishing an object

All the way to rather obscure ways.

```
109 public class ThisEscape {
110     public ThisEscape(EventSource source) {
111         source.registerListener(
112             new EventListener() {
113                 public void onEvent(Event e) {
114                     doSomething(e);
115                 }
116             });
117     }
118 }
```

Publishing an object

All the way to rather obscure ways.

```
119 public class ThisEscape {
120     public ThisEscape(EventSource source) {
121         source.registerListener(
122             new EventListener() {
123                 public void onEvent(Event e) {
124                     doSomething(e);
125                 }
126             });
127     }
128 }
```

```
129 public class ThisEscape
130     public ThisEscape(EventSource source)
131         source.registerListener(
132             new EventListener()
133                 public void onEvent(Event e)
134                     doSomething(e);)
```

After the new keyword, an anonymous class is being created, implementing EventListener.

```
135 public class ThisEscape
136     public ThisEscape(EventSource source)
137         source.registerListener(
138             new EventListener()
139                 public void onEvent(Event e)
140                     doSomething(e);)
```

After the new keyword, an anonymous class is being created, implementing EventListener.

This class is shared to source.registerListener.

```
141 public class ThisEscape
142     public ThisEscape(EventSource source)
143         source.registerListener(
144             new EventListener()
145                 public void onEvent(Event e)
146                     doSomething(e);)
```

After the `new` keyword, an anonymous class is being created, implementing `EventListener`.

This class is shared to `source.registerListener`.

Inner classes carry an implicit reference to `this`.

```
147 public class ThisEscape
148     public ThisEscape(EventSource source)
149         source.registerListener(
150             new EventListener()
151                 public void onEvent(Event e)
152                     doSomething(e);)
```

After the `new` keyword, an anonymous class is being created, implementing `EventListener`.

This class is shared to `source.registerListener`.

Inner classes carry an implicit reference to `this`.

Hence, `this` is shared... in its **constructor** !

this escape (in constructor)

```
153 public class ThisEscape
154     public ThisEscape(EventSource source)
155         source.registerListener(
156             new EventListener()
157                 public void onEvent(Event e)
158                     doSomething(e);)
```

Important!

Do not allow the this reference to escape during construction. Another thread might get a hold of an object before it is properly initialized.

A solution could(!) be to initialize the object in static method.

Pro tip (ad-hoc confinement)

If you can *ensure* that an object always stays within a thread, no synchronization is needed.

Stack confinement

Making sure objects can only be reached through local variables. As local variables are intrinsically tied to the execution of a thread, they cannot escape.

ThreadLocal confinement

Programmatic way to ensure that each thread keeps its own version of the object.

Immutability

One of your most effective tools towards thread-safety.

An *immutable* object is one whose state cannot be changed after its initialization.

Immutability

One of your most effective tools towards thread-safety.

An *immutable* object is one whose state cannot be changed after its initialization.

Important

Immutable objects are always thread-safe.

Immutability

An object is immutable if the following are met:

1. Its state cannot be modified after construction,
2. All its fields are final, and
3. It is properly constructed (the this reference does not escape during construction).

Immutability

Is this object immutable?

```
159 public final class ThreeStooges {
160   private final Set<String> stooges = new HashSet<String>();
161   public ThreeStooges() {
162     stooges.add("Moe");
163     stooges.add("Larry");
164     stooges.add("Curly");
165   }
166   public boolean isStooge(String name) {
167     return stooges.contains(name);
168   }
169 }
```

Immutability

Is this object immutable?

```
170 public final class ThreeStooges {
171   private final Set<String> stooges = new HashSet<String>();
172   public ThreeStooges() {
173     stooges.add("Moe");
174     stooges.add("Larry");
175     stooges.add("Curly");
176   }
177   public boolean isStooge(String name) {
178     return stooges.contains(name);
179   }
180 }
```

Is this object immutable?

```
181 public final class ThreeStooges {
182   private final Set<String> stooges = new HashSet<String>();
183   public ThreeStooges() {
184     stooges.add("Moe");
185     stooges.add("Larry");
186     stooges.add("Curly");
187   }
188   public boolean isStooge(String name) {
189     return stooges.contains(name);
190   }
191 }
```

It does not matter that `stooges` is a `Set` composed of mutable objects. What matters is that stooges itself is `final`.

Important

Just as it is a good practice to make all fields private unless they need greater visibility, it is a good practice to make all fields final unless they need to be mutable.

Important

Just as it is a good practice to make all fields private unless they need greater visibility, it is a good practice to make all fields final unless they need to be mutable.

Tip

Just because an object is immutable, it doesn't mean that it can't *represent* something mutable w.r.t. your program. You can always construct a **new** object with updated values.

Techniques for safe publishing (and sharing)

To publish a (non-immutable) object safely, both the reference to the object and the object's state must be made visible to other threads at the same time. A properly constructed object can be safely published by:

- Initializing an object reference from a static initializer;
- Storing a reference to it into a volatile field or AtomicReference;
- Storing a reference to it into a final field of a properly constructed object; or
- Storing a reference to it into a field that is properly guarded by a lock.

Remember

For objects whose state is going to change after publication, to ensure it's being used properly, it must be either thread-safe or guarded by a lock.

Designing thread-safe classes

The design process for a thread-safe class should include these three basic elements:

Identify the object's *state*, or *state-space*, i.e. its variables.

Identify the **invariants**, meaning inherent properties of the state-space, that constraint its state-space during execution.

Establish a synchronization policy so that concurrent access to the object does not violate its invariants.

The design process for a thread-safe class should include these three basic elements:

Identify the object's *state*, or *state-space*, i.e. its variables.

Identify the **invariants**, meaning inherent properties of the state-space, that constraint its state-space during execution.

Establish a synchronization policy so that concurrent access to the object does not violate its invariants.

Tip

Encapsulation of state is your friend!

Simple invariants


```
192 class SharedCounter {
193     private int count = 0;
194
195     public void increment() {
196         count++;
197     }
198     public int getCount() { return count; }
199 }
```

Simple invariants

```
200 class SharedCounter {
201     private int count = 0;
202
203     public void increment() {
204         count++;
205     }
206     public int getCount() { return count; }
207 }
```

The state of SharedCounter is composed of count, and is already **encapsulated**.

The implicit invariant that an increment should be atomic.

Which policy should be used?

Simple invariants

```
208 class SharedCounter {
209     private int count = 0;
210
211     public synchronized void increment() {
212         count++;
213     }
214     public synchronized int getCount() { return count; }
215 }
```

One idea is to use intrinsic locking to protect the increment operation (will do for now).

Complex invariants


```
216  class NumberRange {
217    private int min, max;
218
219    public void setMin(int newMin) {
220      if (newMin <= max)
221            min = newMin;
222    }
223    public void setMax(int newMax) {
224      if (newMax >= min)
225            max = newMax;
226    }
227    public int getMin() { return min; }
228    public int getMax() { return max; }
229  }
```

Complex invariants

```
230  class NumberRange {
231    private int min, max;
232
233    public void setMin(int newMin) {
234      if (newMin <= max)
235            min = newMin;
236    }
237    public void setMax(int newMax) {
238      if (newMax >= min)
239            max = newMax;
240    }
241    public int getMin() { return min; }
242    public int getMax() { return max; }
243  }
```

Complex invariants

Class `NumberRange` is not thread safe.

No synchronization is used.

The invariants are *implicit*, i.e. no explanation is provided.

Complex invariants (cont)

```
244 class AtomicNumberRange {
245     private final AtomicInteger min = new AtomicInteger(0);
246     private final AtomicInteger max = new AtomicInteger(100);
247
248     public void setMin(int newMin) {
249         if (newMin <= max.get())
250             min.set(newMin);
251     }
252     public void setMax(int newMax) {
253         if (newMax >= min.get())
254             max.set(newMax);
255     }
256     public int getMin() { return min.get(); }
257     public int getMax() { return max.get(); }
258 }
```

Complex invariants (cont)


```
259 class AtomicNumberRange {
260     private final AtomicInteger min = new AtomicInteger(0);
261     private final AtomicInteger max = new AtomicInteger(100);
262
263     public void setMin(int newMin) {
264         if (newMin <= max.get())
265             min.set(newMin);
266     }
267     public void setMax(int newMax) {
268         if (newMax >= min.get())
269             max.set(newMax);
270     }
271     public int getMin() { return min.get(); }
272     public int getMax() { return max.get(); }
273 }
```

Bad delegation

Class `AtomicNumberRange` is not thread safe.

The developer is trying to *delegate* thread safety to elements of the state space of the class itself.

It doesn't work because the two variables are **interdependent**.

Tip

If a class is composed of multiple independent thread-safe state variables and has no operations that have any invalid state transitions, then it can delegate thread safety to the underlying state variables.

Good delegation


```
274 class SafeCounter {
275   private final AtomicInteger count = new AtomicInteger(0);
276
277   public void increment() {
278     count.incrementAndGet();
279   }
280
281   public int getCount() {
282     return count.get();
283   }
284 }
```

Good delegation


```
285 class SafeCounter {
286   private final AtomicInteger count = new AtomicInteger(0);
287
288   public void increment() {
289     count.incrementAndGet();
290   }
291
292   public int getCount() {
293     return count.get();
294   }
295 }
```

The state-space is a single variable, and hence independent.

Complex invariants (cont.)

```
296  class SafeNumberRange {
297    private int min, max;
298
299    public synchronized void setMin(int newMin) {
300      if (newMin <= max)
301            min = newMin;
302    }
303    public synchronized void setMax(int newMax) {
304      if (newMax >= min)
305            max = newMax;
306    }
307    public synchronized int getMin() { return min; }
308    public synchronized int getMax() { return max; }
309  }
```

Complex invariants (cont.)


```
310  class SafeNumberRange {
311    private int min, max;
312
313    public synchronized void setMin(int newMin) {
314      if (newMin <= max)
315            min = newMin;
316    }
317    public synchronized void setMax(
318      if (newMax >= min)
319            max = newMax;
320    }
321    public synchronized int getMin() { return min; }
322    public synchronized int getMax() { return max; }
323  }
```

Encapsulate all shared state.

Use the same lock for **all** access to the shared state.

Tip

Simple, yet effective.

Encapsulate all shared state.

Use the same lock for **all** access to the shared state.

Tip

Simple, yet effective.

Tip

You could also try to use another, internal lock, in case a publicly accessible lock (like in the previous example) may cause liveness issues.

Visibility/Keepsgoing.java: example of visibility issue.

Visibility/KeepsgoingFixed.java: fixed version of the above using sync.

Visibility/Stale.java: more involved example on visibility, with two variables.

Visibility/StaleFixed.java: one (of many) fixed version of the above.

Publishing/ThisEscape*.java: examples on the unsafe publishing of `this`.

Publishing/SafeThisEscape.java: example on the safe publishing of `this`.

Design/GasTankRaceDemo.java: our familiar example on a compound invariant.

Design/GasTankFixed.java: a fixed version of the above.

Also revisited Week10/DataRace.java: example of a Data Race due to operation on `long` being non-atomic.

Thank you!